

[On This Page](#) >

Minimal Example

The `clan vars` subcommand is a powerful tool for managing machine-specific variables in a declarative and reproducible way. This guide will walk you through its usage, from setting up a generator to sharing and updating variables across machines.

For a detailed API reference, see the [vars module documentation](#).

In this guide, you will learn how to:

1. Declare a `generator` in the machine's NixOS configuration.
2. Inspect the status of variables using the Clan CLI.
3. Generate variables interactively.
4. Observe the changes made to your repository.
5. Update the machine configuration.
6. Share the root password between multiple machines.
7. Change the root password when needed.

By the end of this guide, you will have a clear understanding of how to use `clan vars` to manage sensitive data, such as passwords, in a secure and efficient manner.

Declare the generator

In this example, a `vars` `generator` is used to:

- prompt the user for the password
- run the required `mkpasswd` command to generate the hash
- store the hash in a file
- expose the file path to the NixOS configuration

Create a new nix file `root-password.nix` with the following content and import it into your `configuration.nix`

```
1 {config, pkgs, ...}: {  
2  
3   clan.core.vars.generators.root-password = {  
4     # prompt the user for a password  
5     # (`password-input` being an arbitrary name)  
6     prompts.password-input.description = "the root user's password";  
7     prompts.password-input.type = "hidden";  
8     # don't store the prompted password itself  
9     prompts.password-input.persist = false;  
10    # define an output file for storing the hash  
11    files.password-hash.secret = false;  
12    # define the logic for generating the hash  
13    script = ''  
14      cat $prompts/password-input | mkpasswd -m sha-512 > $out/password-h  
15    '';  
16    # the tools required by the script  
17    runtimeInputs = [ pkgs.mkpasswd ];  
18  };  
19  
20  # ensure users are immutable (otherwise the following config might be :  
21  users.mutableUsers = false;  
22  # set the root password to the file containing the hash  
23  users.users.root.hashedPasswordFile =  
24    # clan will make sure, this path exists  
25    config.clan.core.vars.generators.root-password.files.password-hash.pa  
26 }
```

Inspect the status

Executing `clan vars list`, you should see the following:

```
$ clan vars list my_machine  
root-password/password-hash: <not set>
```

...indicating that the value `password-hash` for the generator `root-password` is not set yet.

Generate the values

This step is not strictly necessary, as deploying the machine via `clan machines update` would trigger the generator as well.

To run the generator, execute `clan vars generate` for your machine

```
$ clan vars generate my_machine
Enter the value for root-password/password-input (hidden):
```

After entering the value, the updated status is reported:

```
Updated var root-password/password-hash
  old: <not set>
  new: $6$RMats/YMeypFtcYX$DUi...
```

Observe the changes

With the last step, a new file was created in your repository: `vars/per-machine/my-machine/root-password/password-hash/value`

If the repository is a git repository, a commit was created automatically:

```
1 $ git log -n1
2 commit ... (HEAD -> master)
3 Author: ...
4 Date:    ...
5
6     Update vars via generator root-password for machine grmpf-nix
```

Update the machine

```
clan machines update my_machine
```

Share root password between machines

If we just imported the `root-password.nix` from above into more machines, clan would ask for a new password for each additional machine.

If the root password instead should only be entered once and shared across all machines, the generator defined above needs to be declared as `shared`, by adding `share = true` to it:

```
1 {config, pkgs, ...}: {  
2   clan.core.vars.generators.root-password = {  
3     share = true;  
4     # ...  
5   }  
6 }
```

Importing that shared generator into each machine, will ensure that the password is only asked once the first machine gets updated and then re-used for all subsequent machines.

Change the root password

Changing the password can be done via this command. Replace `my-machine` with your machine. If the password is shared, just pick any machine that has the generator declared.

```
1 $ clan vars generate my-machine --generator root-password --regenerate  
2 ...  
3 Enter the value for root-password/password-input (hidden):  
4 Input received. Processing...  
5 ...  
6 Updated var root-password/password-hash  
7   old: $6$tb27m6E0dff.X9TM$19N...  
8  
9   new: $6$0yoQtDVzeemgh8EQ$zRK...
```

← Previous

Introduction to Vars

Next →

Understanding Clan Vars - Concepts & Architecture

Need help? Reach out to [our community](#).